

DSC650 Final Project: End-to-End Big Data Pipeline for Motorcycle Value Classification

Course Final Project Report

Author: Ayden Trusty | **Institution:** Bellevue University | **Course:** DSC650: Big Data
Instructor: Professor Nasheb Ismaily | **Date:** May 31, 2026

INTRODUCTION

The goal of this project was to build a complete, end-to-end big data pipeline leveraging Apache NiFi, HDFS, Apache Hive, Apache Spark MLlib, and Apache HBase. A motorcycle sales dataset was systematically ingested from GitHub, loaded securely into Hadoop Distributed File System (HDFS), structured and queried using Hive, processed with Spark machine learning components, and the resulting classification evaluation metrics were successfully archived inside HBase.

DATASET

The dataset used for this pipeline contains rich motorcycle sales records capturing distinct attributes including name, selling price, year, seller type, owner history, kilometers driven, and ex-showroom price. The primary target objective of this implementation was to classify motorcycles into high-value and standard-value categories using Spark MLlib framework models.

- **Dataset GitHub Repository:** <https://github.com/TrustyA/DSC650-Big-Data.git>
- **Original Kaggle Source:** <https://www.kaggle.com/datasets/nehalbirla/motorcycle-dataset>

PIPELINE OVERVIEW

The operational topology architectural layout of the engineered pipeline follows the structural framework flow sequence depicted below:

GitHub → Apache NiFi → HDFS → Apache Hive → Apache Spark MLlib → Apache HBase

Apache NiFi orchestrated the automated download of the raw dataset from the version control repository and accurately executed files transfers into HDFS. Apache Hive provided structured data storage layers and optimized analytics capabilities. Spark MLlib trained a predictive Random Forest classification model, and HBase structured columns kept all metrics from model test execution scenarios organized together.

MACHINE LEARNING APPROACH

A `RandomForestClassifier` was selected because the task is an explicit binary classification problem. Motorcycles with a selling price greater than or equal to 100,000 were labeled as high-value motorcycles (1), whereas all other bikes were labeled as standard-value (0). Selected predictor features included vehicle production year, total kilometers driven, ex-showroom base price, seller type index, and ownership history index. The final trained model generated exceptional results, achieving approximately 97% across evaluation benchmarks.

Evaluation Metric	Model Performance Value
Accuracy	0.9718 (97.18%)
Precision	0.9713 (97.13%)
Recall	0.9718 (97.18%)
F1 Score	0.9709 (97.09%)

HBASE & HIVE STRUCTURAL SCHEMA DESIGN

Hive Table Design

The Hive schema was explicitly designed to match the incoming motorcycle sales dataset schema properties accurately. Numeric fields such as `selling_price`, `year`, `km_driven`, and `ex_showroom_price` were explicitly stored using optimized numeric primitives to natively support backend analytical computing and Spark matrix pipelines. Categorical values like `seller_type` and `owner` fields were dynamically processed using indexers down the pipeline.

HBase Architecture

The HBase storage layout utilizes a single column family named `metrics` within the table space `motorcycle_metrics`. A deterministic unique row key of `run_001` maps model testing evaluation metrics together inside a clean atomic horizontal row for rapid analytical indexing.

ISSUES ENCOUNTERED & TROUBLESHOOTING

The largest challenge encountered during production setup involved a Hive schema mismatch resulting from accidentally importing an incorrect motorcycle dataset variation. To correct this, the corrupt table space had to be purged and recreated using a corrected operational schema.

Another issue arose because the Hive command statement `LOAD DATA` naturally moves underlying source files entirely from their native HDFS locations instead of performing a safe file copy action. Consequently, when recreating the database structures, the dataset source files had to be manually reloaded back into the HDFS

directory space before executing a successful secondary load operation. Additional manual terminal troubleshooting steps involved refining environment variables to isolate differences between local Linux OS filesystem paths and absolute cluster HDFS paths.

OBJECTIVES WRITE-UP & VERIFICATION PROOFS

Objective 1 – NiFi Data Ingestion into HDFS

The engineered NiFi dataflow utilizes an `InvokeHTTP` processor block to request the dataset directly from the remote server endpoint, an `UpdateAttribute` processor block to safely standardize underlying file names, and a final `PutHDFS` connector node to deliver data blocks directly to Hadoop clusters.

[Screenshot 1: Apache NiFi Pipeline Workflow Flowchart Graphic]

Screenshot 1

NiFi flow design and processor configurations used to download, rename, and transition data records into HDFS storage space.

[Screenshot 2: Apache NiFi File Content Verification View Terminal Window]

Screenshot 2

NiFi raw content stream verification interface rendering downloaded sales rows and verifying successful pipeline extraction.

[Screenshot 3: Terminal Output - HDFS File System Directory Structure Listing]

Screenshot 3

HDFS storage terminal system validation verifying bike_details.csv file creation within /tmp target folders.

Objective 2 – Hive Managed Table

The structural layout configurations were validated by executing basic analytical counts and grouping aggregation scripts directly across the created managed storage layer table, ensuring record populations match data specifications completely.

[Screenshot 4: Terminal Execution - Hive LOAD DATA Command Statement String]

Screenshot 4

Hive interactive shell execution log confirming successful processing of underlying dataset file resources.

[Screenshot 5: Terminal Execution - Hive SELECT * Table Query Verification Rows]

Screenshot 5

Hive query output parsing raw records directly out of table storage to demonstrate operational schema alignment.

[Screenshot 11: Terminal Execution - Hive MapReduce Aggregation Execution Logs]

Screenshot 11

Hive aggregations and grouping queries utilizing active backend calculations to affirm total row validation constraints.

Objective 3, 4 & 5 – Cluster Preparation, HBase Setup & Spark Processing

To establish a fully unified environment, targeted python libraries including NumPy and HappyBase were distributed and installed across backend cluster nodes. HBase structures were subsequently prepared to map model weights and metrics cleanly.

[Screenshot 6 & 7: Pip Installation Diagnostics Log Output Across Nodes]

Screenshots 6 & 7

Terminal log snapshot demonstrating system-wide installation of necessary communication dependencies.

[Screenshot 8: HBase Table Creation and Shell Scan Diagnostic Step]

Screenshot 8

HBase shell workspace logs highlighting empty initial table space declarations prior to analytical runs.

[Screenshot 9: YARN Master Logs - Spark-Submit Execution Terminal Streaming]

Screenshot 9

Active Spark submission streaming log data showing model generation and final classification report arrays.

Objective 6 & 7 – Pipeline Integration & Storage Verification

Final end-to-end pipeline orchestration was validated via complete cell reads executing directly across HBase server components, confirming that metric evaluations were stored securely.

[Screenshot 10: Terminal Output - HBase Shell Scan Query of Target Tables]

Screenshot 10

HBase scan table query showing successful programmatic row-key generation populated by active Spark scripts.

CONCLUSION

This project successfully implemented and demonstrated an enterprise-grade big data collection pipeline architecture. By coupling NiFi collection topologies, resilient HDFS storage, structural Hive abstraction layers, Spark engine analytical processing models, and column-oriented HBase database frameworks, real-world assets were efficiently extracted, managed, and classified automatically.

APPENDICES

Appendix A – Hive CREATE TABLE Statement

```
CREATE TABLE motorcycles(  
    name STRING,  
    selling_price INT,  
    year INT,  
    seller_type STRING,  
    owner STRING,  
    km_driven INT,  
    ex_showroom_price DOUBLE  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
tblproperties("skip.header.line.count"="1");
```

Appendix B – HBase Commands

```
create 'motorcycle_metrics', 'metrics'  
scan 'motorcycle_metrics'
```

Appendix C – PySpark Pipeline Source Code

```
from pyspark.sql import SparkSession  
from pyspark.sql.functions import when, col  
from pyspark.ml.feature import VectorAssembler, StringIndexer  
from pyspark.ml.classification import RandomForestClassifier  
from pyspark.ml.evaluation import MulticlassClassificationEvaluator  
import happybase  
  
# Create Spark session with Hive support  
spark = (  
    SparkSession.builder  
        .appName("Motorcycle_Value_Classification")  
        .enableHiveSupport()  
        .getOrCreate()  
)  
  
# Load data from Hive table  
df = spark.sql("SELECT name, selling_price, year, seller_type, owner, km_driven,  
ex_showroom_price FROM motorcycles")  
  
# Drop rows with missing important values  
df = df.na.drop()  
  
# Create binary label: 1 = high value motorcycle, 0 = standard value motorcycle  
df = df.withColumn("high_value", when(col("selling_price") >= 100000, 1).otherwise(0))
```



```

# Convert categorical columns into numeric indexes
seller_indexer = StringIndexer(inputCol="seller_type", outputCol="seller_type_index",
handleInvalid="keep")
owner_indexer = StringIndexer(inputCol="owner", outputCol="owner_index",
handleInvalid="keep")

df = seller_indexer.fit(df).transform(df)
df = owner_indexer.fit(df).transform(df)

# Assemble model features
assembler = VectorAssembler(
    inputCols=["year", "km_driven", "ex_showroom_price", "seller_type_index",
"owner_index"],
    outputCol="features",
    handleInvalid="skip"
)
model_df = assembler.transform(df).select("features", "high_value")

# Split training and testing data
train_data, test_data = model_df.randomSplit([0.7, 0.3], seed=42)

# Train Random Forest classifier
rf = RandomForestClassifier(labelCol="high_value", featuresCol="features", numTrees=50,
seed=42)
rf_model = rf.fit(train_data)

# Make predictions
predictions = rf_model.transform(test_data)

# Evaluate model
accuracy_evaluator = MulticlassClassificationEvaluator(labelCol="high_value",
predictionCol="prediction", metricName="accuracy")
precision_evaluator = MulticlassClassificationEvaluator(labelCol="high_value",
predictionCol="prediction", metricName="weightedPrecision")
recall_evaluator = MulticlassClassificationEvaluator(labelCol="high_value",
predictionCol="prediction", metricName="weightedRecall")
f1_evaluator = MulticlassClassificationEvaluator(labelCol="high_value",
predictionCol="prediction", metricName="f1")

accuracy = accuracy_evaluator.evaluate(predictions)
precision = precision_evaluator.evaluate(predictions)
recall = recall_evaluator.evaluate(predictions)
f1 = f1_evaluator.evaluate(predictions)

print("Motorcycle High-Value Classification Results")
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score: {f1}")

# Write metrics to HBase
data = [
    ("run_001", "metrics:accuracy", str(accuracy)),
    ("run_001", "metrics:precision", str(precision)),
    ("run_001", "metrics:recall", str(recall)),
    ("run_001", "metrics:f1", str(f1)),
]

```

```
def write_to_hbase_partition(partition):
    connection = happybase.Connection("master")
    connection.open()
    table = connection.table("motorcycle_metrics")
    for row in partition:
        row_key, column, value = row
        table.put(row_key, {column: value})
    connection.close()

rdd = spark.sparkContext.parallelize(data)
rdd.foreachPartition(write_to_hbase_partition)

spark.stop()
```